

Programming (55 points)

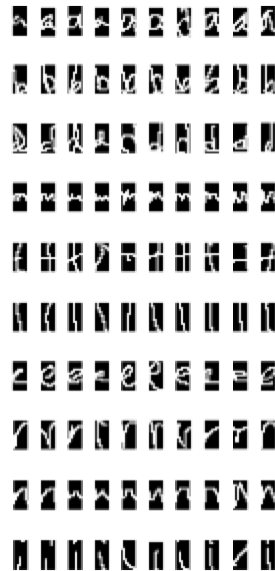


Figure 2: 10 random images of each of the 10 letters in the OCR dataset.

5 The Task

Your goal in this assignment is to implement a neural network to classify images using a single hidden layer neural network.

6 The Datasets

Datasets We will be using a **subset** of an Optical Character Recognition (OCR) dataset. This data includes images of all 26 handwritten letters; our subset will include **only** the letters “a,” “e,” “g,” “i,” “l,” “n,” “o,” “r,” “t,” and “u.” The handout contains a small dataset with 60 samples *per class* (50 for training and 10 for validation). We will also evaluate your code on a medium dataset with 600 samples per class (500 for training and 100 for validation). Figure 2 shows a random sample of 10 images of a few letters from the dataset (not the same ones we’re classifying in this assignment).

File Format Each dataset (small, medium, and large) consists of two csv files—train and validation. Each row contains 129 columns separated by commas. The first column contains the label and columns 2 to 129 represent the pixel values of a 16×8 image in a row major format. Label 0 corresponds to “a,” 1 to “e,” 2 to “g,” 3 to “i,” 4 to “l,” 5 to “n,” 6 to “o,” 7 to “r,” 8 to “t,” and 9 to “u.”

Because the original images are black-and-white (not grayscale), the pixel values are either 0 or 1. However, you should write your code to accept arbitrary pixel values in the range $[0, 1]$. The images in Figure 2 were produced by converting these pixel values into .png files for visualization. Observe that no feature engineering has been done here; instead the neural network you build will *learn* features appropriate for the task of character recognition.

7 Model Definition

In this assignment, you will implement a single-hidden-layer neural network with a sigmoid activation function for the hidden layer, and a softmax on the output layer. Let the input vectors $\mathbf{x}$ be of length M , and the hidden layer $\mathbf{z}$ consist of D hidden units. In addition, let the output layer $\hat{\mathbf{y}}$ be a probability distribution over K classes. That is, each element $\hat{y}_k$ of the output vector represents the probability of $\mathbf{x}$ belonging to the class k .

We can compactly express this model by assuming that $x_0 = 1$ is a bias feature on the input and that $z_0 = 1$ is also fixed. In this way, we have two parameter matrices $\boldsymbol{\alpha} \in \mathbb{R}^{D \times (M+1)}$ and $\boldsymbol{\beta} \in \mathbb{R}^{K \times (D+1)}$. The extra 0th column of each matrix (i.e. $\boldsymbol{\alpha}_{:,0}$ and $\boldsymbol{\beta}_{:,0}$) hold the bias parameters.

$$\begin{aligned} a_j &= \sum_{m=0}^M \alpha_{j,m} x_m \\ z_j &= \sigma(a_j) = \frac{1}{1 + \exp(-a_j)} \\ b_k &= \sum_{j=0}^D \beta_{k,j} z_j \\ \hat{y}_k &= \text{Softmax}(\mathbf{b}) = \frac{\exp(b_k)}{\sum_{l=1}^K \exp(b_l)} \end{aligned}$$

The objective function we will use for training the neural network is the average cross entropy over the training dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}$:

$$J(\boldsymbol{\alpha}, \boldsymbol{\beta}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}) \quad (9)$$

In Equation 9, J is a function of the model parameters $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ because $\hat{y}_k^{(i)}$ is the output of the neural network applied to $\mathbf{x}^{(i)}$ and is therefore implicitly a function of $\mathbf{x}^{(i)}$, $\boldsymbol{\alpha}$, and $\boldsymbol{\beta}$. $\hat{y}_k^{(i)}$ and $y_k^{(i)}$ are the k th components of $\hat{\mathbf{y}}^{(i)}$ and $\mathbf{y}^{(i)}$ respectively.

To train, you should optimize this objective function using stochastic gradient descent (SGD), where the gradient of the parameters for each training example is computed via backpropagation. You should shuffle the training points when performing SGD using the provided `shuffle` function, passing in the epoch number as a random seed. Note that SGD has a slight impact on the objective function as we are “summing” over just the current point, i , and not the entire dataset:

$$J_{SGD}(\boldsymbol{\alpha}, \boldsymbol{\beta}) = -\sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}) \quad (10)$$

You will use the (hopefully at this point) familiar SGD update rule to update the parameters of your model:

$$\alpha_{t+1} \leftarrow \alpha_t - \gamma \frac{\partial J_{SGD}(\alpha_t, \beta_t)}{\partial \alpha_t} \quad (11)$$

$$\beta_{t+1} \leftarrow \beta_t - \gamma \frac{\partial J_{SGD}(\alpha_t, \beta_t)}{\partial \beta_t} \quad (12)$$

where γ is the learning rate, and α_t and β_t are the values of α and β at step t (similarly for α_{t+1} and β_{t+1}).

7.1 Initialization

In order to use a deep network, we must first initialize the weights and biases in the network. This is typically done with a random initialization, or initializing the weights from some other training procedure. For this assignment, we will be using two possible initializations:

RANDOM The weights are initialized randomly from a uniform distribution from -0.1 to 0.1.

The bias parameters are initialized to zero.

ZERO All weights are initialized to 0.

You must support both of these initialization schemes.

8 Implementation

Write a program `neuralnet.py` that implements an optical character recognizer using a one hidden layer neural network with sigmoid activations. Your program should learn the parameters of the model on the training data, report the cross-entropy at the end of each epoch on both train and validation data, and at the end of training write out its predictions and error rates on both datasets.

Your implementation must satisfy the following requirements:

- Use a **sigmoid** activation function on the hidden layer and **softmax** on the output layer to ensure it forms a proper probability distribution.
- Number of **hidden units** for the hidden layer should be determined by a command line flag. (More details on command line flags provided below.)
- Support two different **initialization strategies**, as described in Section 7.1, selecting between them via a command line flag.
- Use stochastic gradient descent (SGD) to optimize the parameters for one hidden layer neural network. The number of **epochs** will be specified as a command line flag.
- Set the **learning rate** via a command line flag.
- Perform stochastic gradient descent updates on the training data on the data shuffled with the provided function. For each epoch, you must reshuffle the **original** file data, not the data from the previous epoch.
- You may assume that the input data will always have the same output label space (i.e. $\{0, 1, \dots, 9\}$). Other than this, do not hard-code any aspect of the datasets into your code. We will autograde your programs on multiple data sets that include different examples.
- In case there is a tie in the output layer $\hat{y}$, predict the smallest index to be the label. (Hint: you shouldn't need to write extra code for tie-breaking if you are using the appropriate NumPy function.)
- Do *not* use any machine learning libraries. You may use NumPy.

Implementing a neural network can be tricky: the parameters are not just a simple vector, but a collection of many parameters; computational efficiency of the model itself becomes essential; the initialization strategy dramatically impacts overall learning quality; other aspects which we will *not* change (e.g. activation function, optimization method) also have a large effect. These tips should help you along the way:

- Try to “vectorize” your code as much as possible—this is particularly important for Python. For example, in Python, you want to avoid for-loops and instead rely on `numpy` calls to perform operations such as matrix multiplication, transpose, subtraction, etc., over an entire `numpy` array at once. Why? Because those calls can be much faster! Those operations are actually implemented in fast C code, which won’t get bogged down the way a high-level scripting language like Python will.
- Implement a finite difference test to check whether your implementation of backpropagation is correctly computing gradients. If you choose to do this, comment out this functionality once your backward pass starts giving correct results and before submitting to Gradescope—since it will otherwise slow down your code.

8.1 Command Line Arguments

The autograder runs and evaluates the output from the files generated, using the following command:

```
$ python3 neuralnet.py [args...]
```

Where above `[args...]` is a placeholder for nine command-line arguments: `<train_input>` `<validation_input>` `<train_out>` `<validation_out>` `<metrics_out>` `<num_epoch>` `<hidden_units>` `<init_flag>` `<learning_rate>`. These arguments are described in detail below:

1. `<train_input>`: path to the training input `.csv` file (see Section 6)
2. `<validation_input>`: path to the validation input `.csv` file (see Section 6)
3. `<train_out>`: path to output `.labels` file to which the prediction on the *training* data should be written (see Section 8.2)
4. `<validation_out>`: path to output `.labels` file to which the prediction on the *validation* data should be written (see Section 8.2)
5. `<metrics_out>`: path of the output `.txt` file to which metrics such as train and validation error should be written (see Section 8.3)
6. `<num_epoch>`: integer specifying the number of times backpropagation loops through all of the training data (e.g., if `<num_epoch>` equals 5, then each training example will be used in backpropagation 5 times).
7. `<hidden_units>`: positive integer specifying the number of hidden units.
8. `<init_flag>`: integer taking value 1 or 2 that specifies whether to use `RANDOM` or `ZERO` initialization (see Section 7.1 and Section 7)—that is, if `init_flag==1` initialize your weights randomly from a uniform distribution over the range `[-0.1, 0.1]` (i.e. `RANDOM`), if `init_flag==2` initialize all weights to zero (i.e. `ZERO`). For both settings, **always initialize bias terms to zero**.
9. `<learning_rate>`: float value specifying the learning rate for SGD.

As an example, if you implemented your program in Python, the following command line would run your program with 4 hidden units on the small data provided in the handout for 2 epochs using zero initialization

and a learning rate of 0.1.

```
python3 neuralnet.py small_train.csv small_validation.csv \
small_train_out.labels small_validation_out.labels \
small_metrics_out.txt 2 4 2 0.1
```

The command line arguments are parsed for you in `neuralnet.py` using the Python builtin `argparse` package.

8.2 Output: Labels Files

Your program should write two output `.labels` files containing the predictions of your model on training data (`<train_out>`) and validation data (`<validation_out>`). Each should contain the predicted labels for each example printed on a new line. Use `\n` to create a new line.

Your labels should exactly match those of a reference implementation – this will be checked by the autograder by running your program and evaluating your output file against the reference solution.

We've included code which outputs correctly formatted labels for you in `neuralnet.py`.

Note: You should output your predicted labels using the same *integer* identifiers as the original training data. You should also insert an empty line (using `'\n'`) at the end of each sequence (as is done in the input data files).

8.3 Output: Metrics

Generate a file where you report the following metrics:

cross entropy After each epoch, report mean cross entropy on the training data `crossentropy(train)` and validation data `crossentropy(validation)` (See Equation 9). These two cross-entropy values should be reported at the end of each epoch and prefixed by the epoch number. For example, after the second pass through the training examples, these should be prefixed by `epoch=2`. The total number of train losses you print out should equal `num_epoch`—likewise for the total number of validation losses.

error After the final epoch (i.e. when training has completed fully), report the final training error `error(train)` and validation error `error(validation)`.

A sample output for the small data set is given below. It contains the train and validation losses for the first 2 epochs and the final error rate output by the command given at the end of section 8.1 Command Line Arguments.

```
epoch=1 crossentropy(train): 2.1415670910950144
epoch=1 crossentropy(validation): 2.1502231738985618
epoch=2 crossentropy(train): 1.8642629963917074
epoch=2 crossentropy(validation): 1.8780601379038728
error(train): 0.73
error(validation): 0.72
```

Take care that your output has the exact same format as shown above. There is an equal sign `=` between the word `epoch` and the epoch number, but no spaces. There should be a single space after the epoch number (e.g. a space after `epoch=1`), and a single space after the colon preceding the metric value (e.g. a space

after `epoch=1 crossentropy(train):`). Each line should be terminated by a Unix line ending `\n`. We've include code which correctly formats your metrics for you in `neuralnet.py`.

8.4 Unit Tests

To help you debug your code, we've included a unit test file in your handout, `tests.py`. This is a *nonexhaustive* set of unit tests which are meant to help you make sure your implementation is correct. Passing these tests does not guarantee a full score in your Gradescope submission, but it will help you identify functions which have errors. Do not edit these tests as we will not be able to guarantee correctness if you modify these tests.

To run the unit tests, run the following command lines:

```
To run one test: python -m unittest tests.TestRandomInit.test_shape  
To run one set of tests: python -m unittest tests.TestRandomInit  
To run all tests: python -m unittest tests
```

If the above commands give you errors, try replacing `python` with `python3`.

9 Gradescope Submission

You should submit your `neuralnet.py` to Gradescope. **Any other files will be deleted.** Please do not use any other file name for your implementation. This will cause problems for the autograder to correctly detect and run your code.

Make sure to read the autograder output carefully. The autograder for Gradescope prints out some additional information about the tests that it ran. For this programming assignment we've specially designed some buggy implementations that you might implement and will try our best to detect those and give you some more useful feedback in Gradescope's autograder. Make wise use of autograder's output for debugging your code.

Note: For this assignment, you may make up to **10** submissions to Gradescope before the deadline, but only your last submission will be graded.

10 Module-Based Neural Net Implementation

10.1 Module-based Method of Implementation

Module-based automatic differentiation (AD) is a technique that has long been used to develop libraries for deep learning, and is the method of implementation that you are encouraged to follow in this assignment. Dynamic neural network packages are those that allow a specification of the computation graph dynamically at runtime, such as Torch¹, PyTorch², and DyNet³—these all employ module-based AD in the sense that we will describe here.⁴

The key idea behind module-based AD is to componentize the computation of the neural-network into layers. Each layer can be thought of as consolidating numerous nodes in the computation graph (a subset of them) into one *vector-valued* node. Such a vector-valued node should be capable of the following and we call each one a **module** (corresponding to a class in Python):

1. Forward computation of output $\mathbf{b} = [b_1, \dots, b_B]$ given input $\mathbf{a} = [a_1, \dots, a_A]$ via some differentiable function f . That is, $\mathbf{b} = f(\mathbf{a})$.
2. Backward computation of the gradient of the input $\mathbf{g}_\mathbf{a} = \frac{\partial J}{\partial \mathbf{a}} = [\frac{dJ}{da_1}, \dots, \frac{dJ}{da_A}]$ given the gradient of output $\mathbf{g}_\mathbf{b} = \frac{\partial J}{\partial \mathbf{b}} = [\frac{dJ}{db_1}, \dots, \frac{dJ}{db_B}]$, where J is the final real-valued output of the entire computation graph. This is done via the chain rule $\frac{dJ}{da_i} = \sum_{j=1}^B \frac{dJ}{db_j} \frac{\partial b_j}{\partial a_i}$ for all $i \in \{1, \dots, A\}$.

10.1.1 Module Definitions

In our implementation, the modules we will define for our neural network correspond to a Linear layer and a Sigmoid layer. While it is possible to additionally define modules for Softmax and Cross-Entropy, we keep them as functions for simplicity (though you are welcome to turn them into modules as well if you wish). Each module defines a forward method $\mathbf{b} = \text{*.FORWARD}(\mathbf{a})$, and a backward method $\mathbf{g}_\mathbf{a} = \text{*.BACKWARD}(\mathbf{g}_\mathbf{b})$. In other words, the forward method yields the output, $\mathbf{b}$, given the input, $\mathbf{a}$; meanwhile, the backward method yields the gradient with respect to the input, $\mathbf{g}_\mathbf{a}$, given the gradient with respect to the output, $\mathbf{g}_\mathbf{b}$. Each module may also store certain values as appropriate (for instance, the Linear layers store the weight matrices α, β).

Note that for linear modules in particular, while the gradients with respect to the inputs and outputs are passed in and out of the modules, the gradients with respect to the weight matrices, $\mathbf{g}_\alpha$ and $\mathbf{g}_\beta$ are **not**. This follows the object-oriented design principle of encapsulation – $\mathbf{g}_\alpha$ and $\mathbf{g}_\beta$ are only required by their respective linear layers, so we only store them within the linear module itself. Later on, they will be used for a SGD update, which will be performed by an additional STEP method. (Alternatively, since the SGD update for this assignment is always applied per example, you may directly perform the SGD update within BACKWARD, though you should be extra careful about the order of your operations.)

Further, if you've completed Written Question 2, you might notice that though we only pass $\mathbf{g}_\mathbf{b}$, the gradient with respect to the module output, into $\text{*.BACKWARD}(\mathbf{g}_\mathbf{b})$, we may need more than that to calculate some of the layer's gradients. Specifically, if you inspect your expressions for the gradient, you may notice that they use certain values from the forward pass. Hence, in your forward methods, you will want to **cache** certain values to be used later on in the backward pass. In the starter code, we do so via a `cache` dictionary

¹<http://torch.ch/>

²<http://pytorch.org/>

³<https://dymnet.readthedocs.io>

⁴Static neural network packages are those that require a static specification of a computation graph which is subsequently compiled into code. Examples include Theano, Tensorflow, and CNTK. These libraries are also module-based but the particular form of implementation is different from the dynamic method we recommend here.

as a class attribute, wherein you can store parameter names as keys that map to their cached values.

Finally, you'll want to pay close attention to the dimensions that you pass into and return from your modules. The dimensions A and B are specific to the module such that we have input $\mathbf{a} \in \mathbb{R}^A$, output $\mathbf{b} \in \mathbb{R}^B$, gradient of output $\mathbf{g}_\mathbf{a} \triangleq \nabla_{\mathbf{a}} J \in \mathbb{R}^A$, and gradient of input $\mathbf{g}_\mathbf{b} \triangleq \nabla_{\mathbf{b}} J \in \mathbb{R}^B$.

We have provided you the pseudocode for the Linear Module as an example.

Linear Module

```

1: procedure FORWARD( $\mathbf{a}$ )
2:   Compute  $\mathbf{b}$  using this layer's weight matrix
3:   Cache intermediate value(s) for the backward pass           ▷ See Written Question 1.2(d)
4:   return  $\mathbf{b}$ 
5: procedure BACKWARD( $\mathbf{g}_\mathbf{b}$ )
6:   Bring the necessary cached values into scope
7:   Compute  $\mathbf{g}_\alpha$ 
8:   Compute  $\mathbf{g}_\mathbf{a}$ 
9:   Store  $\mathbf{g}_\alpha$  for subsequent SGD update
10:  return  $\mathbf{g}_\mathbf{a}$ 
11: procedure STEP()
12:  Apply SGD update to weights  $\alpha$  using stored gradient  $\mathbf{g}_\alpha$ 

```

10.1.2 Module-based AD for Neural Network

Given that our one hidden layer neural network is a composition of modules, we can define functions for forward and backward propagation using these modules as follows:

Algorithm 1 Forward Computation

```

1: procedure NNFORWARD(Training example ( $\mathbf{x}, \mathbf{y}$ ))
2:    $\mathbf{a} = \text{LINEAR1.FORWARD}(\mathbf{x})$            ▷ First linear layer module
3:    $\mathbf{z} = \text{SIGMOID.FORWARD}(\mathbf{a})$          ▷ Sigmoid activation module
4:    $\mathbf{b} = \text{LINEAR2.FORWARD}(\mathbf{z})$          ▷ Second linear layer module
5:    $\hat{\mathbf{y}} = \text{SOFTMAX}(\mathbf{b})$              ▷ Softmax function
6:    $J = \text{CROSSENTROPY}(\mathbf{y}, \hat{\mathbf{y}})$        ▷ CrossEntropy function
7:   return  $J, \hat{\mathbf{y}}$ 

```

Algorithm 2 Backpropagation

```

1: procedure NNBACKWARD( $\mathbf{y}, \hat{\mathbf{y}}$ )
2:    $g_J = \frac{\partial J}{\partial J} = 1$            ▷ Base case
3:    $\mathbf{g}_\mathbf{b} = \text{DSOFTMAXCROSSENTROPY}(\mathbf{y}, \hat{\mathbf{y}}, g_J)$    ▷ See Written Question 1.2(b)
4:    $\mathbf{g}_\mathbf{z} = \text{LINEAR2.BACKWARD}(\mathbf{g}_\mathbf{b})$ 
5:    $\mathbf{g}_\mathbf{a} = \text{SIGMOID.BACKWARD}(\mathbf{g}_\mathbf{z})$ 
6:    $\mathbf{g}_\mathbf{x} = \text{LINEAR1.BACKWARD}(\mathbf{g}_\mathbf{a})$            ▷ We discard  $\mathbf{g}_\mathbf{x}$ 

```

Here's the big takeaway: we can actually view these two functions as themselves defining another module! It is a 1-hidden layer neural network module. That is, the cross-entropy of the neural network for a single

training example is *itself* a differentiable function and we know how to compute the gradients of its inputs, given the gradients of its outputs.

10.2 Training Procedure

Consider the neural network described in Section 7 applied to the i th training example $(\mathbf{x}, \mathbf{y})$ where $\mathbf{y}$ is a one-hot encoding of the true label. Our neural network outputs $\hat{\mathbf{y}} = h_{\alpha, \beta}(\mathbf{x})$, where α and β are the parameters of the first and second layers respectively and $h_{\alpha, \beta}$ is a one-hidden layer neural network with a sigmoid activation and softmax output. The loss function is negative cross-entropy $J = \ell(\hat{\mathbf{y}}, \mathbf{y}) = -\mathbf{y}^T \log(\hat{\mathbf{y}})$. $J = J_{\mathbf{x}, \mathbf{y}}(\alpha, \beta)$ is actually a function of our training example $(\mathbf{x}, \mathbf{y})$ as well as our model parameters α, β , though we write just J for brevity.

In order to train our neural network, we are going to apply stochastic gradient descent (SGD). Because we want the behavior of your program to be approximately deterministic for testing on Gradescope, we will require that (1) you should use *our provided* shuffle function to shuffle your data at the start of each epoch and (2) you will use a fixed learning rate.

SGD proceeds as follows, where E is the number of epochs and γ is the learning rate.

Algorithm 3 Training with Stochastic Gradient Descent (SGD)

```

1: procedure SGD(Training data  $\mathcal{D}_{train}$ , test data  $\mathcal{D}_t$ )
2:   Initialize parameters  $\alpha, \beta$  ▷ Use either RANDOM or ZERO from Section 7.1
3:   for  $e \in \{1, 2, \dots, E\}$  do ▷ For each epoch
4:      $\mathcal{D} = \text{SHUFFLE}(\mathcal{D}_{train}, e)$ 
5:     for  $(\mathbf{x}, \mathbf{y}) \in \mathcal{D}$  do ▷ For each training example
6:       Compute neural network forward prop:
7:        $J, \hat{\mathbf{y}} = \text{NN.FORWARD}(\mathbf{x}, \mathbf{y}, \alpha, \beta)$ 
8:       Compute gradients via backprop:
9:        $\left. \begin{array}{l} \mathbf{g}_\alpha = \frac{\partial J}{\partial \alpha} \\ \mathbf{g}_\beta = \frac{\partial J}{\partial \beta} \end{array} \right\} \text{ given by NN.BACKWARD}(\mathbf{y}, \hat{\mathbf{y}})$ 
10:      Update parameters with SGD updates  $\mathbf{g}_\alpha, \mathbf{g}_\beta$ :
11:       $\alpha \leftarrow \alpha - \gamma \mathbf{g}_\alpha$ 
12:       $\beta \leftarrow \beta - \gamma \mathbf{g}_\beta$ 
13:      Evaluate training mean cross-entropy  $J_{\mathcal{D}}(\alpha, \beta)$ 
14:      Evaluate test mean cross-entropy  $J_{\mathcal{D}_t}(\alpha, \beta)$ 
15:   return parameters  $\alpha, \beta$ 

```

10.3 Testing Procedure

At test time, we output the most likely prediction for each example:

Algorithm 4 Prediction at Test Time

```

1: procedure PREDICT(Unlabeled train or test dataset  $\mathcal{D}'$ )
2:   for  $\mathbf{x} \in \mathcal{D}'$  do
3:     Compute neural network prediction  $\hat{\mathbf{y}} = h(\mathbf{x})$ 
4:     Predict the label with highest probability  $l = \text{argmax}_k \hat{y}_k$ 

```
